Practice midterm 1 (with solutions)

TC1037 Algorithmic Thinking · Group 510 · Tecnológico de Monterrey, Campus Querétaro

Practice — download, solve on paper in 80 minutes, then check the solutions

Name: _____ **Matrícula:** _____

80 minutes · 100 points · individual · on paper. No computer, phone, notes or calculator. Write in pen or dark pencil; code must be legible and indented. **Part I** (30 pts): write *exactly* what Python prints — every line, in order. **Part II** (70 pts): write the program. Suggested time: 15 minutes for Part I, 65 for Part II. **This is a practice paper: same format, same difficulty, different problems.**

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	5	5	5	5	5	5	23	23	3	20	1	100
Score:												

Part I — Read and predict **(30 points)**

1. **Arithmetic operators.** What does Python print? [5]

```
print(6 + 9 // 4 * 3 - 7 % 2 ** 3)
```

Solution: `5`

Why

Order: `**` first, then `*` `/` `//` `%` left to right, then `+` `-` left to right.

`6 + 9 // 4 * 3 - 7 % 2 ** 3`

= `6 + 9 // 4 * 3 - 7 % 8` — `**`: $2^3 = 8$

= `6 + 2 * 3 - 7 % 8` — `//`: 4 fits in 9 two whole times

= `6 + 6 - 7 % 8` — `*`

= `6 + 6 - 7` — `%`: 8 fits in 7 zero times, 7 is left over

= `5` — no `/` anywhere, so the result is an integer, not `5.0`

2. **Boolean operators.** What does Python print? [5]

```
print(not 2 >= 2 or 6 / 3 == 2 and 1 < 0)
```

Solution: `False`

Why

Order: arithmetic, then comparisons, then `not`, then `and`, then `or`.

`not 2 >= 2 or 6 / 3 == 2 and 1 < 0`

= `not 2 >= 2 or 2.0 == 2 and 1 < 0` — arithmetic: `/` gives `2.0`

= `not True or True and False` — comparisons (`2.0 == 2` is True: same value)

= `False or True and False` — `not`

= `False or False` — `and` before `or`: True `and` False is False

= `False`

3. **Conditional structure.** What does Python print? [5]

```
x = 20
y = 6
if x % y == 2:
    print("A", x // y)
elif x > 10:
    print("B", x % y)
else:
    print("C")
print("end")
```

Solution: `A 3`
`end`

Why

`x % y` = `20 % 6` = 2 (6 fits three times in 20, 2 left over), so the `if` is True and prints `A` and `20 // 6` = 3.
The `elif x > 10` is also True — but it is **never checked**: in an `if/elif/else` only the *first* True branch runs.
`print(ënd)` is outside the `if`, so it always runs. Output: `A 3` then `end`.

4. **Functions.** What does Python print? [5]

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)

main()
```

Solution: `2 3 15`

Why

Arguments go **by position, not by name**: `h(y, x)` sends main's `y` (3) into h's `x`, and main's `x` (2) into h's `y`.

Inside h: `x = 3 + 2` = 5; `y = 5 - 2` = 3; `return 5 * 3` = 15.

h's `x` and `y` are *its own* variables: changing them never changes main's. Back in main, `x` is still 2, `y` still 3, `z` is 15 → `2 3 15`.

5. **The while loop.** What does Python print? [5]

```
n = 50
count = 0
while n > 5:
    n = n // 2
    count = count + 1
print(n, count)
```

Solution: `3 4`

Why

One row per turn; the test is checked *before* each turn.

turn	`n > 5`?	`n`	`count`
start	—	50	0
1	yes	25	1
2	yes	12	2
3	yes	6	3
4	yes	3	4
5	no		

`25 // 2` = 12 (the .5 is dropped). `3 > 5` is False, so the loop stops and prints the last row: `3 4`.

6. **The for loop.** What does Python print? [5]

```
for i in range(1, 12, 4):
    print(i * i)
```

Solution:
```
1
25
81
```

Why

`range(1, 12, 4)`: start at 1, jump by 4, stop *before* 12 → `i` takes 1, 5, 9 (the next one, 13, would pass 12).
The `print` is inside the loop, so one line per turn: $1^2 = 1$, $5^2 = 25$, $9^2 = 81$.

Part II — Write the program **(70 points)**

Write **one** Python program with the following parts. All `input()` and all validation happen in `main()`; the two functions **return** their result and never print or read. **Function 7 must use a `while` loop and function 8 a `for` loop** — each is worth 23 points. Write each function's purpose in a one-line comment above it, and under each loop a one-line comment saying why it stops (`while`) or how many times it runs (`for`).

7. **`while loop`** The function `sum_digits(n)` receives a non-negative integer `n` and **returns** the sum of its digits. Use a `while` loop with `%` and `//`. [23]

   Examples: `sum_digits(1234)` → `10` `sum_digits(9)` → `9` `sum_digits(1000)` → `1` `sum_digits(0)` → `0`

8. **`for loop`** The function `count_multiples(n, k)` receives two integers $n \geq 1$ and $k \geq 1$ and **returns** how many multiples of k there are from 1 to n, inclusive. Use a `for` loop. [23]

   Examples: `count_multiples(10, 3)` → `3` `count_multiples(12, 3)` → `4` `count_multiples(5, 7)` → `0` `count_multiples(7, 7)` → `1`

9. The function `menu()` that displays: `1. sum_digits  2. count_multiples  3. Exit` [3]

10. The function `main()` that, inside a `while` loop: shows the menu, reads the option, and [20]
    - option 1: reads n (must be 0 or more); if any value is out of range displays a clear message, otherwise calls `sum_digits` and displays the result with a label;
    - option 2: reads n and k (both must be 1 or more); same rule, calling `count_multiples`;
    - option 3: displays `Goodbye.` and the program ends;
    - any other option: displays `Invalid option.` and the program ends.

    Use `if/elif/else` to choose the option.

11. The last line of the script calls `main()`. [1]

Technical requirements — 2 points off for each one missing: meaningful variable names in Python style · a clear prompt for every input · readable, labelled output.

Reference solution

```python
# Reference solution — Practice midterm 1 (with solutions)

def sum_digits(n):
    total = 0
    while n > 0:
        total = total + n % 10
        n = n // 10
    return total


def count_multiples(n, k):
    count = 0
    for i in range(1, n + 1):
        if i % k == 0:
            count = count + 1
    return count
```

```python
def menu():
    print("1. sum_digits")
    print("2. count_multiples")
    print("3. Exit")


def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            n = int(input("Number n (0 or more): "))
            if n < 0:
                print("Invalid input: n must be 0 or more.")
            else:
                print("Sum of the digits:", sum_digits(n))
        elif option == 2:
            n = int(input("n (1 or more): "))
            k = int(input("k (1 or more): "))
            if n < 1 or k < 1:
                print("Invalid input: n and k must be 1 or more.")
            else:
                print("Multiples of", k, "up to", n, ":", count_multiples(n, k))
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:
            print("Invalid option.")
            keep_going = False


main()
```

How to think about `sum_digits`

Accumulator `total` starts at 0. Each turn: `n % 10` is the last digit (add it), `n // 10` removes it. The loop stops because `n` loses a digit every turn until it is 0.

Check the smallest input: `sum_digits(0)` → the loop never runs → returns 0. Correct. **Always test the smallest input**: in other problems it is exactly where a plain `while n > 0` goes wrong.

How to think about `count_multiples`

`range(1, n + 1)` visits 1, 2, …, n; `i % k == 0` means `i` is a multiple of `k`.

The classic slip: `range(1, n)` never reaches `n`, so `count_multiples(12, 3)` gives 3 instead of 4 (12 itself is a multiple). That is *Muy bien*, not *Excelente*.

How to think about `main`

A flag `keep_going` controls the `while`. Every option follows the same four steps: **read** (`int(input(…))`) → **validate** (`if` out of range → message) → **call** the function → **print with a label**.

Option 3 and any invalid option set the flag to False, so the loop ends. Validation is done in `main`, never inside the function: the function only computes and `returns`.

How Part II is graded (the academia's rubric)

Each part of the program (the two functions, `menu`, `main`, the call) is scored on this scale, as a percentage of its points:

Excelente 100%	**Muy bien 85%**	**Regular 70%**	**Deficiente 50%**	**Muy deficiente 20%**	**Sin evidencia 0%**
No syntax errors; works correctly, including the boundary cases in the examples.	Correct approach; minimal logic or syntax slips.	At most two syntax errors and/or small logic errors; the approach is closely related to what was asked.	Approach loosely related, or related but with more than two syntax errors.	Code unrelated to the problem.	No code.

Part I: 5 points per question, all or nothing — every line of the output, in order. Technical requirements: -2 each, from Part II.